

REQMEMBENCH: BENCHMARKING CODING AGENTS ON TEMPORAL REQUIREMENT-STATE RECONSTRUCTION

Anonymous authors

Paper under double-blind review

ABSTRACT

Coding agents increasingly join ongoing software projects, where they must modify an evolving codebase in the context of a long-running client collaboration. Before acting, an agent must determine which requirements currently govern the task. This requires tracing how requirements have changed over time and determining which still apply, while distinguishing changes to the requirements themselves from evidence about their implementation. Existing coding and memory benchmarks offer limited visibility into whether agents correctly reconstruct the requirement state that should guide implementation. We introduce ReqMemBench, a benchmark for temporal requirement-state reconstruction built from real freelance project histories that have been rewritten for privacy and include a subset of role-played messages. At intermediate takeover points, ReqMemBench provides evidence-linked gold requirement states and evaluates four capabilities: selecting relevant historical requirements and evidence, reconstructing their pre-task states, updating requirements or identifying material clarification needs, and delivering the requested code changes. A two-phase protocol freezes history-based responses before agents choosing to act on eligible tasks receive repository access. Full History and Oracle Relevant History conditions assess the effect of history filtering and agents’ reconstruction accuracy when relevant evidence is explicitly supplied. [TO COMPLETE: report the principal quantitative findings, including performance under oracle history and the relationship between requirement-state accuracy and code delivery.]

1 INTRODUCTION

Code-generation evaluation has progressively expanded from self-contained function synthesis to repository-scale software engineering. HumanEval and MBPP evaluate short programs generated from natural-language specifications (Chen et al., 2021; Austin et al., 2021). CrossCodeEval and RepoBench introduce cross-file dependencies and repository context (Ding et al., 2023; Liu et al., 2024), while SWE-bench evaluates patches for real GitHub issues and SWE-Lancer evaluates freelance software-engineering tasks using executable or expert-derived criteria (Jimenez et al., 2024; Miserendino et al., 2025). These benchmarks have substantially increased the scope and realism of coding evaluation. However, they generally assume that the task to be implemented is adequately specified by the current issue or prompt. An agent joining an ongoing project faces an additional prerequisite: before modifying the code, it must determine what the project currently requires.

This prerequisite motivates our central research question: *Can a coding agent reconstruct the requirements that currently govern a task from a long and evolving project history, determine whether those requirements are sufficiently resolved to act, and implement the task accordingly?* Answering this question requires more than retrieving relevant messages. In a long-running project, requirements may be introduced, refined, deferred, resumed, or removed. A later update may alter only part of an earlier requirement, while execution feedback may change the known implementation status without changing the requirement itself. Consequently, previously authoritative statements may no longer apply in full, yet may still provide essential evidence about how the current requirement emerged. The agent must integrate these updates to recover the task-relevant requirement state at

takeover, including the currently applicable constraints, their scope and lifecycle status, unresolved ambiguities, and available evidence about their implementation.

Several lines of work evaluate capabilities related to this problem. Conversational coding benchmarks examine how models revise programs in response to iterative execution, or natural-language feedback (Han et al., 2025). Long-term memory benchmarks evaluate information extraction, temporal reasoning, knowledge updates, long-range understanding, selective forgetting, and abstention over extended interactions (Wu et al., 2025; Maharana et al., 2024; Hu et al., 2025). These benchmarks test whether agents can use historical information, and some additionally evaluate retrieval quality. However, they do not explicitly treat the current task-relevant requirement state as a gold evaluation target. As a result, they cannot separately determine whether an unsuccessful implementation arose because the agent selected the wrong history, reconstructed the requirements incorrectly, failed to recognize a material ambiguity, or implemented a correctly understood requirement poorly.

We formulate this problem as *temporal requirement-state reconstruction* and introduce ReqMemBench to evaluate it. Given the interaction history preceding a takeover point and a new request, an agent first reconstructs the pre-task state of each directly affected requirement immediately before the request. It must then incorporate the new request and either produce the complete post-task requirement states or identify ambiguities that materially prevent safe implementation. ReqMemBench derives evidence-linked gold states from annotated, transformed freelance project histories and evaluates four capabilities: history selection, state reconstruction, update-or-clarify reasoning, and code delivery. To assess historical reasoning independently of repository access, the protocol freezes the agent’s history-based response before eligible action runs receive the pre-task repository. Full History and Oracle Relevant History compare performance with the complete transformed history and an audited relevant subsequence that preserves the evidence needed to determine the same gold targets. Our contributions are:

- We formulate temporal requirement-state reconstruction for coding-agent takeover, with explicit targets for pre-task states, post-task updates, and material clarification needs.
- We construct ReqMemBench from transformed longitudinal freelance project histories, linking requirement events and temporal states to their supporting evidence at intermediate takeover points.
- We provide a two-phase evaluation protocol that separately scores history-based requirement reasoning and repository delivery, with paired history conditions for examining the effect of oracle history filtering.

2 RELATED WORK

Coding benchmarks. Coding benchmarks cover function-level synthesis and library use (Chen et al., 2021; Austin et al., 2021; Zhuo et al., 2025), cross-file completion (Ding et al., 2023; Liu et al., 2024), repository issue resolution and freelance engineering (Jimenez et al., 2024; Miserendino et al., 2025), and release-level evolution and project development (Le et al., 2025; Lu et al., 2026). Sequential benchmarks address knowledge transfer, trajectory reuse, and code-quality degradation under evolving specifications (Joshi et al., 2025; Zhu et al., 2026; Orlanski et al., 2026). RigorBench additionally evaluates engineering process discipline, including abstention and clarification (Madiraju & Madiraju, 2026).

Figure 1 placeholder: Motivation and task definition

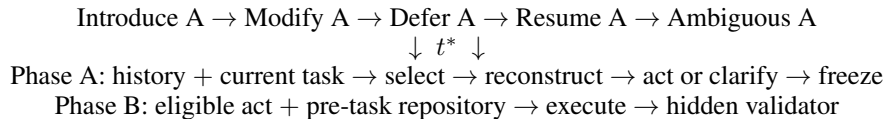


Figure 1: The ReqMemBench takeover setting. An agent first reasons from the preceding history and current task without repository access. After its response is frozen, eligible ACT runs receive the pre-task repository for delivery evaluation.

Memory and long context. LongBench and RULER evaluate retrieval and reasoning over extended inputs (Bai et al., 2024; Hsieh et al., 2024). LongMemEval, MemoryAgentBench, and LoCoMo assess complementary capabilities across interactions, including temporal and multi-session reasoning, knowledge updates, selective forgetting, and abstention (Wu et al., 2025; Hu et al., 2025; Maharana et al., 2024). These evaluations cover both access to historical information and its integration and revision.

Conversational coding. ConvCodeWorld and RECODE-H evaluate iterative code revision under execution or simulated expert feedback (Han et al., 2025; Miao et al., 2026), while SR-Eval studies stepwise requirement refinement (Zhan et al., 2025). LoCoEval introduces conflicting requirement information in repository-oriented conversations and separately evaluates information-item extraction and function generation at conversation end (Liu et al., 2026). ReqMemBench focuses on task-local requirement states at intermediate takeover points, explicitly tracking attributes, lifecycle, scope, ambiguity, and implementation evidence. Its protocol freezes reconstruction and update-or-clarify responses before repository access and evaluates them alongside eligible code delivery.

3 TEMPORAL REQUIREMENT-STATE RECONSTRUCTION

Requirements evolve through revisions, deferrals, resumptions, and removals, so a takeover agent must recover the state that governs the project now rather than merely retrieve earlier statements. Building on requirements traceability (Nuseibeh & Easterbrook, 2000; Gotel & Finkelstein, 1994), ReqMemBench represents this state explicitly and evaluates its reconstruction before code execution.

3.1 TEMPORAL STATE AND TASK-LOCAL GOLD

We represent a project as $P = (H, C)$, where $H = \langle h_1, \dots, h_n \rangle$ is the ordered interaction history and $C(t)$ is the corresponding code state. A target request q_{t^*} defines a strict temporal boundary: $H_{<t^*}$ and C_{t^*-} contain only pre-target evidence. We use t^{*+} for the point after incorporating requirement changes expressed by the target but before any subsequent implementation or execution feedback.

The atomic unit is a *Requirement Atom*. For atom r , its state is

$$G_r(t) = (A_r(t), L_r(t), S_r(t), U_r(t), X_r(t)), \quad (1)$$

where A_r records attribute values, L_r the lifecycle, S_r persistence and scope, U_r unresolved ambiguities, and X_r the latest implementation evidence. Unknown fields remain explicit. These dimensions are independent: an active requirement can be ambiguous or have failed implementation evidence, while a removed requirement retains its prior attributes and evidence under lifecycle REMOVED.

States are reconstructed by deterministically replaying the validated event sequence E_r up to time t :

$$G_r(t) = \text{Replay}(E_r^{\leq t}), \quad (2)$$

using the transition rules in the appendix. Let $R_P(t)$ be the atoms introduced by time t , and let A_{t^*} be all atoms affected by the target, including newly introduced ones. The task-local pre- and post-target gold states are

$$\begin{aligned} R_{t^*}^{\text{hist}} &= A_{t^*} \cap R_P(t^{*-}), \\ G_{t^*}^- &= (G_r(t^{*-}))_{r \in R_{t^*}^{\text{hist}}}, \\ G_{t^*}^+ &= (G_r(t^{*+}))_{r \in A_{t^*}}. \end{aligned} \quad (3)$$

Thus, atoms introduced by the target are absent from $G_{t^*}^-$ but included in $G_{t^*}^+$. The task-local projection evaluates selection, reconstruction, and updating without requiring recovery of unrelated project state.

3.2 TAKEOVER PROTOCOL AND EVALUATION STAGES

Full History (C1) exposes $H_{<t^*}$, whereas Oracle Relevant History (C2) exposes an audited, order-preserving subsequence containing the evidence required to determine the same task-local state and

Table 1: The four evaluation stages. C1 is Full History and C2 is Oracle Relevant History. RQ4 applies only to eligible targets.

Stage	Evaluated capability	Gold target	History	Repository
RQ1	Select affected historical atoms and supporting evidence	$R_{t^*}^{\text{hist}}$ and supporting evidence groups	C1	No
RQ2	Reconstruct pre-task states	$G_{t^*}^-$ restricted to matched atoms	C1/C2	No
RQ3	Update requirements or identify material blockers	$G_{t^*}^+$ for ACT; blocking issues and questions for CLARIFY	C1/C2	No
RQ4	Implement the requested changes	Final repository passing the frozen validator	C1/C2	After freeze

action. C2 isolates reasoning over relevant evidence from the additional problem of finding that evidence in a long history.

Evaluation separates history reasoning from code execution. In Phase A, the agent receives the condition-specific history and current request, but no repository:

$$(H_{<t^*}^{(c)}, q_{t^*}) \longrightarrow (\hat{R}_{t^*}^{\text{hist}}, \hat{G}_{t^*}^-, \hat{d}_{t^*}, \hat{Z}_{t^*}), \quad (4)$$

where $c \in \{C1, C2\}$ and $\hat{d}_{t^*} \in \{\text{ACT}, \text{CLARIFY}\}$. The output contains selected historical atoms and evidence, their reconstructed pre-task states, and either the complete predicted post-task state $\hat{G}_{t^*}^+$ for ACT or material blocking issues and questions $\hat{Q}_{t^*}$ for CLARIFY. Clarification is correct only when visible evidence leaves a target-relevant ambiguity that blocks a determinate update.

The Phase-A response is frozen before repository access. For an execution-eligible target, Phase B begins only after a parseable ACT response, at which point the agent receives a fresh copy of C_{t^*} . Hidden validators remain inaccessible to the agent, separating requirement-state prediction from repository correctness.

RQ1 is evaluated only under C1 because C2 is selected from the gold requirement trajectories and would therefore reveal the evidence-selection target. The remaining stages use both conditions, allowing errors to be localized across reconstruction, decision, and execution.

4 REQMEMBENCH CONSTRUCTION

ReqMemBench turns longitudinal project records into takeover tasks whose governing requirements can be reconstructed and tested. Construction proceeds from the project timeline to reviewed Requirement Events, from those Events to temporal Requirement States, and finally from selected takeover points to paired agent inputs and gold targets. Figure 2 summarizes this process. The definitions in Section 3 determine what each instance must contain; the construction pipeline makes those objects recoverable from project evidence.

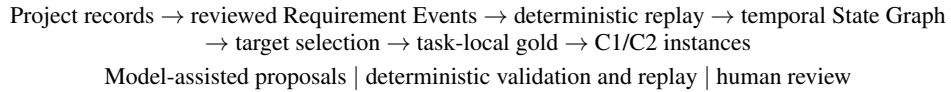


Figure 2: ReqMemBench converts longitudinal project records into evidence-linked Requirement States and leakage-controlled takeover instances.

4.1 TEMPORAL REQUIREMENT GRAPH CONSTRUCTION

Construction begins with longitudinal records of real freelance software projects. We transform the project histories to protect participant privacy while preserving the order and meaning of requirement changes. When an exact historical message or executable environment is unavailable, we reconstruct it from the surviving project evidence and the requirement state at the corresponding

time. Instance-level provenance distinguishes transformed records, reconstructed messages, recovered environments, and reconstructed environments. The Ethics Statement and Data Card specify the authorization, transformation, and release procedures for these records.

The transformed timeline is then converted into the representation defined in Section 3. A model first proposes Requirement Atoms, state-changing Events, and links from each Event to its supporting messages. Schema checks reject malformed objects and broken references, after which human review resolves semantic errors and freezes the accepted annotations. Deterministic replay applies the reviewed Events in project order to obtain the state of every Requirement Atom after each change. An invalid transition stops replay instead of being repaired by another model call, and each resulting state retains the evidence from which it was derived. The complete annotation schema and transition rules appear in the appendix. *[TODO: report frozen project and message counts and the proportions of transformed histories, reconstructed messages, recovered code environments, and reconstructed code environments.]*

4.2 TAKEOVER INSTANCE CONSTRUCTION

The temporal graph becomes a benchmark instance only at a task that both depends on earlier project context and produces a verifiable Requirement transition. All changes requested in the same client message remain within one takeover task. We exclude context-free introductions and messages that provide implementation evidence without changing a requirement. The remaining candidates pass through a fixed historical-dependence and requirement-evolution rubric, followed by human review for borderline cases. The appendix gives the rubric, threshold, and candidate flow.

For an accepted target, deterministic replay is stopped immediately before the target and resumed through the target message. These two boundaries yield the pre-task and post-task states defined in Section 3. Although the graph retains the complete project state, the instance contains the task-local projection over affected Requirements. A Requirement introduced by the target therefore appears only in the post-task state, whereas a removed Requirement remains in the projection with lifecycle REMOVED. This alignment produces the gold objects used to evaluate historical selection, state reconstruction, and update-or-clarify decisions.

For each accepted target, we materialize the paired history conditions defined in Section 3.2. The oracle history is extracted from the pre-target timeline using annotated requirement trajectories and their supporting context, then audited for preservation of the task-local gold. Each pair shares the current request and gold targets; executable pairs also share the pre-task repository and hidden validator. Agent-visible inputs are serialized separately from gold annotations, internal identifiers, future messages, and evaluation artifacts. *[TODO: report accepted and rejected targets, C1/C2 instance counts, and RQ4 candidate, environment, eligibility, and exclusion counts.]*

4.3 CURATION VALIDITY AND BENCHMARK STATISTICS

Validity checks follow the same boundaries as construction. Code validates schemas, references, event order, and deterministic replay, while human reviewers decide the semantic content of Requirement Events and ambiguity-sensitive gold. The annotation audit draws a stratified sample of at least 200 state transitions. A human reviewer and two independent LLM assessors label each sampled transition under the same annotation scheme; raw agreement and a coefficient matched to the final rater design quantify consistency. Disagreements are adjudicated before the benchmark snapshot is frozen. This annotation audit is distinct from calibrating the RQ1 semantic judge, adjudicating RQ3 decisions, and validating RQ4 repositories.

A separate audit tests whether Oracle Relevant History preserves the information needed for evaluation. Its messages must form an ordered subsequence of Full History and support the same pre-task state and RQ3 gold target. If the two views yield different gold interpretations, the oracle subset is corrected; targets that cannot be repaired without adding unavailable evidence are excluded. This prevents curation errors from appearing as an experimental effect of history selection.

The appendix characterizes the corpus by Requirement lifecycle, Event type, provenance category, repository recoverability, and exclusion reason. *[TODO: insert agreement and oracle-history audit results, correction counts, and regenerated benchmark statistics after the review freeze.]*

Table 2: ReqMemBench corpus statistics at the current construction snapshot. Evaluation counts are finalized after annotation review and eligibility checks.

Level	Main-text statistic	Current snapshot
Project	Projects	51
Requirement	Requirement Atoms	859
Event	Requirement Events	2,793
Task	Takeover tasks and source projects	210 tasks; 39 projects
Evaluation	Reviewed/eligible instances by RQ and condition	[TBD]

5 EXPERIMENTS

We evaluate whether a coding agent can carry an evolving specification from historical evidence to a validated repository. The evaluation unit is a model–harness pair, and the logical run unit is a target–condition pair. One frozen Phase-A response supplies all applicable RQ1–RQ3 outputs. Phase B begins only when the RQ3 decision is ACT and the target passes the RQ4 eligibility gate. This design preserves the dependency between the four capabilities without rerunning the same agent separately for each research question.

5.1 EXPERIMENTAL SETUP

Experimental controls. We follow the evaluation protocol in Section 3.2. For each target, C1 and C2 use the same current request, model–harness configuration, prompt, and response schema, with identical tools and resource budgets within each phase. Each condition and replicate runs in a fresh workspace without cross-run session memory, and condition labels are hidden from the agent. Paired execution comparisons use the same pre-task repository and hidden validator. We analyze performance across three history-length strata: Short (0–25 prior turns), Medium (26–50), and Long (> 50).

Agents and run controls. We treat the model and its coding-agent harness as a single deployed system because scaffolding, tool use, and stopping behavior can affect the result. The experiment manifest will record the provider model identifier and revision, harness version, evaluation date, context window, temperature and sampling settings, prompt and output-schema hashes, permitted tools and network access, token and wall-clock budgets, retry and stopping policies, and the number of independent runs. The runner will retain the frozen Phase-A response, tool and token logs, final Phase-B repository, and evaluator outputs. *[TODO: freeze the evaluated Codex and Claude Code configurations, exact model versions, prompts, budgets, evaluation dates, and replicate policy before formal runs.]*

Scoring. RQ1 aligns predicted and gold requirement atoms through a frozen semantic-relation classifier followed by deterministic one-to-one matching. It reports Requirement Precision, Recall, and F1; Evidence Precision, Recall, and F1; and Exact Requirement Set Accuracy. Requirement and evidence scores remain separate, and RQ1 receives no C2 result.

RQ2 evaluates the pre-task state only for one-to-one matched historical requirements. Typed comparators score attributes, scope, lifecycle, ambiguity, and execution. *Matched Full-State Exact* requires every applicable field of every matched requirement to be correct. Because unmatched gold requirements are not scored a second time, Reconstruction Coverage, $|M_t|/|R_t^{\text{gold}}|$, is reported separately. Targets with no matched requirement are N/A, not perfect reconstructions. An oracle-aligned constant-state baseline exposes class imbalance. The paired C2–C1 contrast assesses the effect of oracle history filtering. We additionally compare C1 and C2 on gold requirements matched in both conditions. We report the numbers of targets and requirements included in this comparison.

RQ3 first scores the ACT/CLARIFY decision using Decision Accuracy, Balanced Accuracy, and class recall. Gold-ACT targets additionally report typed Post-State Score, Post-State Exact, and end-to-end success. A missing affected requirement scores zero, and an extra requirement is a closed-

Table 3: Primary metrics and comparison sets for the four evaluation stages.

Stage	Main-text metrics	Scoring set or comparison
RQ1	Requirement/Evidence F1; Exact Set	C1 targets; target-level macro
RQ2	Attribute Reconstruction; Matched Full-State Exact; Coverage	Matched historical requirements; paired C2–C1
RQ3	Decision/Balanced Accuracy; ACT and CLARIFY success	Separate Gold-ACT and Gold-CLARIFY denominators
RQ4	Repository Success Rate; Executable Coverage	Eligible targets; C1/C2 common support

world false positive. Gold-CLARIFY targets report blocking-issue Precision/Recall/F1, question validity, and clarification success, with requirement-, dimension-, and field-level diagnostics. Un-supported Autonomy uses Gold-CLARIFY targets as its denominator, whereas Unnecessary Clarification uses Gold-ACT targets. All-ACT and all-CLARIFY decision baselines are reported for both conditions.

RQ4 scores only the final repository. A target-condition pair is eligible only if its frozen RQ3 gold decision is ACT, the pre-task environment is reproducibly runnable, the requested behavior is deterministically observable, the hidden target tests have passed two independent reviews, the validator has passed calibration, and the agent-visible repository passes the leakage audit. For an eligible target,

$$\text{RQ4Pass} = \text{BuildPass} \wedge \text{TargetTestPass} \wedge \text{RegressionPass}.$$

A frozen non-ACT or unparsable Phase-A response on an eligible target yields NO_CODE_SUBMISSION/FAIL; evaluator, environment, or harness failures instead invalidate the attempt and trigger a clean rerun. RQ4 reports Success Rate and Executable Coverage. The primary C1/C2 comparison uses only targets eligible under both conditions with the same validator, while all-eligible results and coverage are shown separately.

Aggregation and uncertainty. Metrics are first computed at their stated unit and then macro-averaged over targets; benchmark-level cross-project claims use project macro-averages so that projects with many targets do not dominate. Every table reports the number of targets for each condition and RQ3 branch. C2–C1 effects use paired targets with identical gold, and RQ4 uses the common-support set described above. Class-conditional recall and risk rates with small denominators will include their numerator, denominator, and two-sided exact-binomial 95% confidence interval. Semantic-judge or schema failures are recorded as JUDGE_ERROR and rerun rather than converted into agent errors. Detailed field scores, judge calibration, validator calibration, and per-project results will be reported in the appendix. *[TODO: freeze the replicate aggregation rule, confidence-interval and paired-comparison procedures, multiple-comparison policy, and final handling of agent timeouts and other missing outputs.]*

5.2 MAIN RESULTS: REQUIREMENT IDENTIFICATION AND RECONSTRUCTION

RQ1: Can agents select the relevant historical requirements? *[Writing template: Begin with one sentence that answers RQ1 using Requirement F1 and its uncertainty under C1. Identify the strongest model-harness pair and quantify the gap between Requirement F1, Evidence F1, and Exact Requirement Set Accuracy. Then explain the dominant precision or recall failure with one representative case. Report all target counts and avoid generalizing from a single project or history-length stratum.]*

RQ2: Can agents reconstruct the current requirement state? *[Writing template: Begin with one sentence that answers RQ2 using Attribute Reconstruction and Matched Full-State Exact. Report the paired C2–C1 difference with uncertainty, followed by Reconstruction Coverage so that matched-only state quality is not mistaken for end-to-end selection performance. Compare against the oracle-aligned constant-state baseline, identify the weakest state dimension, and discuss a counterexample. Do not apply current-task updates to the pre-task state.]*

Table 4: Main-result template for RQ1 selection and RQ2 pre-task reconstruction. All entries remain TBD until the formal evaluation is frozen and executed.

Panel A: RQ1 selection under C1									
Model/agent	Targets	Req. P	Req. R	Req. F1	Exact Set	Evidence P	Evidence R	Evidence F1	
[Model/agent]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]

Panel B: RQ2 reconstruction under C1/C2									
Model/agent	Cond./contrast	<i>N</i>	Attributes	Scope	Lifecycle	Ambiguity	Execution	Matched Exact	Recon. Coverage
[Model/agent]	C1	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
[Model/agent]	C2	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
[Model/agent]	C2–C1	[paired TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
Oracle-aligned constant	Baseline	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]

Table 5: Main-result template for RQ3 update-or-clarify reasoning and RQ4 repository delivery. All entries remain TBD until the formal evaluation is frozen and executed.

Panel A: RQ3 decision quality									
Model/agent	Cond.	Gold A/C	Dec. Acc.	Bal. Acc.	ACT Rec.	CLARIFY Rec.	Unsup. Autonomy	Unnec. Clarify	
[Model/agent]	C1	[TBD/TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
[Model/agent]	C2	[TBD/TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
All-ACT	C1/C2	[TBD/TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
All-CLARIFY	C1/C2	[TBD/TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]

Panel B: RQ3 Gold-ACT branch						
Model/agent	Cond.	Gold-ACT <i>N</i>	Post-State Score	Post-State Exact	End-to-End Success	
[Model/agent]	C1	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
[Model/agent]	C2	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]

Panel C: RQ3 Gold-CLARIFY branch											
Model/agent	Cond.	Gold-CLARIFY <i>N</i>	Req. Correct	Dim. Correct	Field Correct	Blocker P	Blocker R	Blocker F1	Question Validity	Clarification Success	
[Model/agent]	C1	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
[Model/agent]	C2	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]

Panel D: RQ4 repository delivery							
Model/agent	Cond.	Gold-ACT <i>N</i>	Eligible <i>N</i>	Common <i>N</i>	PASS (common)	PASS (eligible)	Exec. Coverage
[Model/agent]	C1	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
[Model/agent]	C2	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]

5.3 MAIN RESULTS: UPDATE DECISIONS AND CODE DELIVERY

RQ3: Can agents produce the correct requirement update or clarification? [Writing template: Answer RQ3 first with Decision Accuracy and Balanced Accuracy under C1/C2, together with Gold ACT/CLARIFY counts and the all-ACT/all-CLARIFY baselines. Report paired C2–C1 effects before separating the branches. For Gold-ACT targets, report Post-State Exact and end-to-end success; for Gold-CLARIFY targets, report Blocking-Issue F1, Question Validity, and Clarification Success. State the numerator, denominator, and exact 95% interval for Unsupported Autonomy and Unnecessary Clarification, and do not combine their denominators.]

RQ4: Can agents translate requirements into correct code? [Writing template: Answer RQ4 with repository Success Rate on the C1/C2 common-support set, including paired uncertainty. Then report each condition’s all-eligible Success Rate and Executable Coverage with Gold-ACT, eligible, and common-support counts. Attribute a history-condition effect only on common support. Reserve Build, Target Test, and Regression pass rates for failure analysis, and do not report formal PASS/FAIL values until the RQ3 gold, eligibility gates, validators, calibration, and leakage audits are frozen.]

6 ANALYSIS AND LIMITATIONS

This section will analyze concrete failure mechanisms rather than only model rankings. Each implication will be tied to an observed result or case.

Table 6: Planned cross-stage failure analysis.

Stage	Diagnostic question	Typical failure
RQ1	Were the affected historical requirements and evidence selected?	Missed gold atom/evidence group or imported unrelated requirement
RQ2	Were the matched pre-task states reconstructed?	Stale attributes, widened scope, ignored life-cycle, or confused execution evidence
RQ3	Was the complete update or every material blocker identified?	Incorrect update, unsupported action, or unnecessary/mislocalized question
RQ4	Did the eligible ACT run pass the frozen validator?	Correct reasoning followed by Build, Target Test, or Regression failure

Figure 4 placeholder: cross-stage error propagation

Conditional RQ4 PASS rates given correct versus incorrect RQ2/RQ3 outputs, optionally paired with the strongest verified history-characteristic effect.

Figure 3: Planned diagnostic of how requirement-state and decision errors propagate to repository delivery.

6.1 FAILURE MODES AND ERROR PROPAGATION

Requirement-evolution failure modes. The analysis will cover `INTRODUCE`, `MODIFY`, `DEFER`, `RESUME`, `REMOVE`, `AMBIGUOUS`, `IMPLEMENTATION_CLAIM`, `RUNTIME_FAILURE`, and `RUNTIME_VERIFICATION`. `CLARIFY` will be analyzed as an RQ3 decision rather than a Requirement Event. Initial categories include stale-state errors that retain superseded values, resurrection errors that reuse removed requirements, missing-scope errors, ambiguity hallucinations, and execution-state confusion. *[TODO: revise this taxonomy using observed errors and add anonymized, provenance-safe examples.]*

Effects of history characteristics. Analyze history length, evidence distance, relevant-evidence density, number of updates, parallel requirements, target position, and number of affected requirements. The final analysis will specify bins, controls, model, and multiple-comparison treatment before interpreting correlations.

Does unfiltered history hurt? Compare C1 Full History with C2 Oracle Relevant History on paired targets with the same gold. A C2–C1 difference will be attributed to removal of irrelevant and stale context only after confirming identical prompts, models, budgets, non-repository tools, and sufficient oracle evidence. Candidate mechanisms include noise, stale-state interference, truncation, context competition, and dimension-specific state errors. No-History remains outside the formal comparison.

Coding ability versus requirement-state ability. Compare model rankings on a standard coding benchmark, ReqMemBench pre/post-state reasoning, and ReqMemBench RQ4 delivery. Any ranking inversion will be interpreted only after aligning model versions, evaluation dates, harnesses, and uncertainty. If no inversion occurs, the final text will report the observed relationship directly.

Error propagation across the four stages. The analysis distinguishes

MemoryFailure $\neq$ StateReasoningFailure $\neq$ DecisionFailure $\neq$ ImplementationFailure.

6.2 IMPLICATIONS AND LIMITATIONS

Implications for coding-agent design. The final discussion will retain only implications supported by measured evidence. Candidate implications are that memory should represent current state rather than only retrieve messages; invalidation can matter as much as remembering; requirement memory should preserve temporal provenance; material unresolved ambiguity should trigger

clarification; and requirement reasoning should be frozen before repository execution so code cannot reveal the state-reconstruction answer.

Limitations. The final paper will cover:

1. **Data domain.** Freelance projects do not represent every enterprise process, organization, safety requirement, or codebase.
2. **Data-generation provenance.** If histories or code states are rewritten, reconstructed, simulated, or mixed, conclusions apply to that construction rather than to unqualified naturally occurring development.
3. **Historical takeover setting.** ReqMemBench supplies an observed history and does not directly evaluate an online memory-writing policy from t_0 .
4. **Annotation subjectivity.** Requirement atomicity, typed state fields, material ambiguity, post-task branches, and clarification targets permit interpretation differences and require stage-specific reliability evidence.
5. **Code availability and executability.** Dependencies, environments, external services, or original snapshots may not be recoverable for every project.
6. **Requirement gold versus implementation gold.** Multiple implementations may satisfy one requirement:

$$\text{RequirementGold} \neq \text{UniqueCodePatch}.$$
7. **Evaluation-judge dependence.** Semantic scorers, target selectors, and validators may introduce model or schema bias; calibration and deterministic checks bound but do not eliminate this risk.
8. **Sampling and dependence.** Multiple targets from one project are correlated, and empty or imbalanced history strata limit generalization.
9. **Coverage.** The benchmark may not cover multi-team enterprise development, combined issue-tracker/chat/code-review histories, multi-year memory, or continuously operating autonomous agents.

7 CONCLUSION

The conclusion will briefly state what existing evaluation misses, what ReqMemBench introduces, and the one or two most important measured findings. ReqMemBench evaluates whether an agent can identify directly affected historical requirements, reconstruct their pre-task states, update or clarify affected post-task states, and complete eligible repository deliveries at intermediate takeover points. *[TODO: insert the final findings and end with the supported form of the central message: persistent coding agents must know both how to code and what the project requires now.]*

AI USE STATEMENT

[TODO: complete this statement according to the current ICLR 2027 author policy; disclose generative-AI use in research ideation, data processing, annotation, code, experiments, writing, or editing; and explain human verification and responsibility.]

ETHICS STATEMENT

[TODO: describe data authorization, Terms-of-Service or contractual constraints, privacy and de-identification, credentials and sensitive content, reconstructed or simulated artifacts, release scope, re-identification risk, potential harms, and safeguards. Distinguish source-data access from the public release.]

REPRODUCIBILITY STATEMENT

[TODO: point to the sections, appendices, code, configurations, prompts, schemas, dataset cards, and supplementary artifacts covering selection, provenance, PII processing, annotation, replay, tar-

get construction, instance materialization, models, metrics, judge calibration, validator calibration, and statistics.]

REFERENCES

- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021. URL <https://arxiv.org/abs/2108.07732>. MBPP.
- Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. LongBench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 3119–3137, 2024. doi: 10.18653/v1/2024.acl-long.172. URL <https://aclanthology.org/2024.acl-long.172/>.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. URL <https://arxiv.org/abs/2107.03374>. HumanEval.
- Yangruibo Ding, Zijian Wang, Wasi Uddin Ahmad, Hantian Ding, Ming Tan, Nihal Jain, Murali Krishna Ramanathan, Ramesh Nallapati, Parminder Bhatia, Dan Roth, and Bing Xiang. CrossCodeEval: A diverse and multilingual benchmark for cross-file code completion. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, volume 36, 2023. doi: 10.52202/075280-2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/920f2dcdd7d32ab2ba2f1970bc306af6-Abstract-Datasets_and_Benchmarks.html.
- O. C. Z. Gotel and Anthony Finkelstein. An analysis of the requirements traceability problem. In *Proceedings of the First IEEE International Conference on Requirements Engineering*, pp. 94–101. IEEE Computer Society, 1994. doi: 10.1109/ICRE.1994.292398.
- Hojae Han, Seung-won Hwang, Rajhans Samdani, and Yuxiong He. ConvCodeWorld: Benchmarking conversational code generation in reproducible feedback environments. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/6091f2bb355e960600f62566ac0e2862-Abstract-Conference.html.
- Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. RULER: What’s the real context size of your long-context language models? In *Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=kIoBbc76Sy>.
- Yuanzhe Hu, Yu Wang, and Julian McAuley. Evaluating memory in LLM agents via incremental multi-turn interactions. *arXiv preprint arXiv:2507.05257*, 2025. URL <https://arxiv.org/abs/2507.05257>. MemoryAgentBench.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*,

2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/edac78c3e300629acfe6cbe9ca88fb84-Abstract-Conference.html.
- Thomas Joshi, Shayan Chowdhury, and Fatih Uysal. SWE-Bench-CL: Continual learning for coding agents. *arXiv preprint arXiv:2507.00014*, 2025.
- Tue Le, Minh Vu Thai Pham, Dung Nguyen Manh, Huy Nhat Phan, and Nghi D. Q. Bui. SWE-EVO: Benchmarking coding agents in long-horizon software evolution scenarios. *arXiv preprint arXiv:2512.18470*, 2025. v6 revised May 2026 – CHECK: set year to 2026 if you cite the revision or a subsequent venue version.
- Tianyang Liu, Canwen Xu, and Julian McAuley. RepoBench: Benchmarking repository-level code auto-completion systems. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/d191ba4c8923ed8fd8935b7c98658b5f-Abstract-Conference.html.
- Yang Liu, Li Zhang, Fang Liu, Ping Lin, and Xinyi Li. A scalable benchmark for repository-oriented long-horizon conversational context management. *arXiv preprint arXiv:2603.06358*, 2026. LoCoEval.
- Pengrui Lu, Shiqi Zhang, Yunzhong Hou, Lyumanshan Ye, Chaoyi Huang, Zixi Chen, Ji Zeng, Hantao Jiang, Pengfei Liu, Yiwei Wang, and Ming-Hsuan Yang. ProjDevBench: Benchmarking AI coding agents on end-to-end project development. *arXiv preprint arXiv:2602.01655*, 2026.
- Meher Bhaskar Madiraju and Meher Sai Preetam Madiraju. RigorBench: Benchmarking engineering process discipline in autonomous AI coding agents. *arXiv preprint arXiv:2606.22678*, 2026. Unrefereed preprint, 30 tasks – see notes on citation weight.
- Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 13851–13870. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.acl-long.747. URL <https://aclanthology.org/2024.acl-long.747/>.
- Chunyu Miao, Henry Peng Zou, Yangning Li, Yankai Chen, Yibo Wang, et al. RECODE-H: A benchmark for research code development with interactive human feedback. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2510.06186.
- Samuel Miserendino, Michele Wang, Tejal Patwardhan, and Johannes Heidecke. SWE-Lancer: Can frontier LLMs earn \$1 million from real-world freelance software engineering? In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 44412–44450. PMLR, 2025. URL <https://proceedings.mlr.press/v267/miserendino25a.html>.
- Bashar Nuseibeh and Steve M. Easterbrook. Requirements engineering: A roadmap. In *Proceedings of the 22nd International Conference on Software Engineering, Future of Software Engineering Track*, pp. 35–46. ACM, 2000. doi: 10.1145/336512.336523.
- Gabriel Orlanski, Devjeet Roy, Alexander Yun, Changho Shin, Alex Gu, Albert Ge, Dyah Adila, Nicholas Roberts, Frederic Sala, and Aws Albarghouthi. SlopCodeBench: Benchmarking how coding agents degrade over long-horizon iterative tasks. *arXiv preprint arXiv:2603.24755*, 2026.
- Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/d813d324dbf0598bbdc9c8e79740ed01-Abstract-Conference.html.
- Zexun Zhan, Shuzheng Gao, Ruida Hu, and Cuiyun Gao. SR-Eval: Evaluating LLMs on code generation under stepwise requirement refinement. *arXiv preprint arXiv:2509.18808*, 2025.

Jiayuan Zhu, Junde Wu, Minhao Hu, Shengda Zhu, Jiazhen Pan, Weixiang Shen, Yijun Yang, Fenglin Liu, Jianye Hao, Yueming Jin, Qirong Ho, and Min Xu. SWE Context Bench: A benchmark for context learning in coding. *arXiv preprint arXiv:2602.08316*, 2026. Official name is "SWE Context Bench"; not the same paper as ContextBench (arXiv:2602.05892).

Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, et al. BigCodeBench: Benchmarking code generation with diverse function calls and complex instructions. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2406.15877.

A APPENDIX STRUCTURE

The appendix will contain:

1. the complete requirement-state schema and field definitions;
2. the event taxonomy and state-transition rules;
3. project selection, provenance categories, authorization, privacy, and de-identification procedures;
4. annotation guidelines, review interfaces, and agreement analysis;
5. target-time selection and leakage controls;
6. expanded RQ1–RQ4 definitions and examples, complete prompts, public response schemas, and hidden scoring contracts;
7. model, agent, tool, budget, and retry configurations;
8. full metric equations, field-level comparators, branch-specific denominators, aggregation, uncertainty, and statistical tests;
9. judge and validator calibration, baselines, and human-ceiling estimates;
10. full benchmark distributions and eligibility counts;
11. full per-RQ result tables and breakdowns by project, event, condition, model, and history characteristic;
12. failure cases and qualitative examples;
13. one end-to-end example from source evidence through events, the state graph, target-local pre/post states, the frozen Phase-A output and score, and validated repository delivery; and
14. a data card and release statement explaining which artifacts can and cannot be shared.